

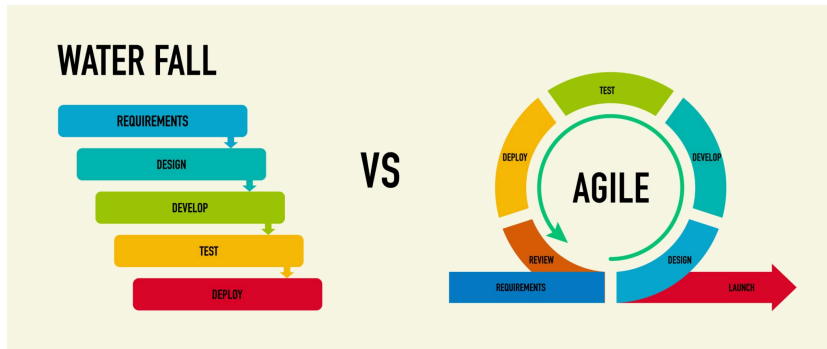
SDLC vs. ADLC: Agile vs. Traditional

<https://gemini.google.com/app/93b129a5bce7de18>

User prompt: Okay now lets btalk about sdlc and adlc

Response: The debate between traditional **SDLC** (Software Development Life Cycle) and **ADLC** (Agile Development Life Cycle) is basically the difference between building a bridge and building a startup.

With one, you need everything planned down to the millimeter before you pour any concrete. With the other, you build a basic version, see if people can cross it, and keep widening and reinforcing it as you go.



1. Traditional SDLC (The Mapmaker)

Traditional SDLC (often associated with the **Waterfall** model) is highly structured and sequential. You don't move to the next phase until the previous one is 100% completed, signed off, and locked in.

- **How it works:** You spend months gathering requirements, design the entire system architecture, write all the code, test it at the very end, and finally deploy.
- **The Big Risk:** If you discover a major flaw or a shift in user needs during the testing phase, going backward to fix it is incredibly expensive and slow.

2. ADLC (The Explorer)

ADLC is Agile's spin on the development cycle. Instead of treating software like a physical construction project, ADLC treats it as an evolving organism.

- **How it works:** You break the project down into tiny, functional chunks (sprints). Every 1 to 4 weeks, you go through a mini-lifecycle: plan, design, build, test, and deliver a working slice of software.
- **The Big Advantage:** You get real user feedback almost immediately. If a feature isn't working out, you pivot next week, not next year.

Side-by-Side Comparison

Feature	Traditional SDLC (Waterfall)	ADLC (Agile)
Structure	Linear and sequential	Iterative and circular
Requirements	Locked in at the start	Expected to change and evolve
Testing	Happens at the end of the project	Continuous (done during every sprint)

Feature	Traditional SDLC (Waterfall)	ADLC (Agile)
Risk Management	High risk (untested assumptions accumulate)	Low risk (issues caught early in small cycles)
Delivery	One massive "Big Bang" launch	Regular, incremental updates

Which one is actually better? Neither is objectively "better" overall, but they shine in different scenarios. If you are writing software for a pacemaker or a space shuttle, you want traditional SDLC—mistakes are not an option. If you are building a new mobile app or SaaS platform where speed-to-market and user feedback are everything, ADLC is the way to go.

User prompt: explain SDLC only in detail also about waterfall, Alternative and Agile\

Response: To understand software engineering, you have to start with the baseline blueprint: the **Software Development Life Cycle (SDLC)**.

No matter what philosophy or buzzword a team uses (Waterfall, Agile, DevOps), *every* piece of software goes through these exact same core steps. The only thing that changes is **how** we cycle through them.

Part 1: The Core SDLC Phases (In Detail)

Think of the SDLC as the universal step-by-step pipeline for taking a rough idea and turning it into running, maintained code.

1 Planning & Feasibility

Phase 1

This is the "Should we even build this?" phase. Business analysts, project managers, and stakeholders define the project's scope, calculate the budget (ROI), assess risks, and determine if the team has the technical capability to build it.

2 Requirements Analysis

Phase 2

The goal here is absolute clarity on *what* the software must do. Analysts compile a **Software Requirement Specification (SRS)** document. This outlines both **functional requirements** (e.g., "users can log in via Google") and **non-functional requirements** (e.g., "the page must load under 2 seconds").

3 Design & Architecture

Phase 3

Architects and lead developers draft the blueprints. This is split into **Logical Design** (data flow, database schemas, and system relationships) and **Physical Design** (hardware choices, programming languages, and API structures).

4 Coding & Implementation

Phase 4

This is the actual building phase. Developers take the design documents and write the code. They follow coding standards and use version control tools (like Git) to collaborate. This is often the most time-consuming phase.

5 Testing & Quality Assurance

Phase 5

Before the public sees the code, QA engineers try to break it. They check for bugs, security vulnerabilities, and ensure the code matches the original SRS document. Testing ranges from automated unit tests to manual exploratory testing.

6 Deployment & Release

Phase 6

Once the software is deemed stable, it's released to the production environment where real users can access it. This can be a massive, single-day rollout or a staged release (beta testing) to a small percentage of users.

7 Maintenance & Operations

Phase 7

Real-world use always reveals unexpected bugs, security threats, or performance issues. This ongoing phase involves patch updates, server monitoring, and minor feature tweaks to keep the software running smoothly.

Part 2: The Three Main SDLC Methodologies

While the core phases above never change, **how we move through them** defines our development methodology. Let's look at the three major approaches.

1. The Waterfall Model (Sequential)

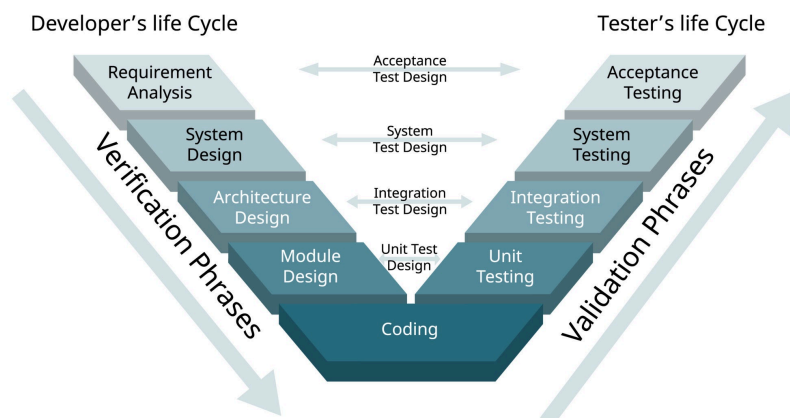
The Waterfall model is the grandfather of software engineering. It is a **strict linear process**. You flow down from Phase 1 to Phase 7 like water over a cliff—once you drop down a step, there is no going back up.

- **The Workflow:** You cannot start coding (Phase 4) until the entire Design (Phase 3) is signed off. You cannot test (Phase 5) until 100% of the code is written.
- **Best For:** Projects where requirements are set in stone and cannot change (e.g., medical devices, aerospace engineering, or banking systems).
- **The Catch:** It is incredibly rigid. If you find a requirement flaw during testing, you practically have to restart the entire project.

2. Alternative Models (Bridging the Gap)

As projects grew more complex, teams realized Waterfall was too rigid. Several "alternative" models emerged to introduce flexibility without losing structure:

- **The V-Model (Verification & Validation):** An extension of Waterfall, but with a twist. For every development phase on the left side of the "V", there is a corresponding testing phase planned on the right side. It ensures QA is considered at every single step, rather than just tacked on at the end. (*See the V-Model visualization below*).
- **The Spiral Model:** A risk-driven model. It splits the project into four quadrants: planning, risk analysis, engineering, and evaluation. The team repeatedly loops through these quadrants, building increasingly complex prototypes with each spiral. It's excellent for massive, high-risk projects.
- **Iterative / Incremental Model:** Instead of building the entire system at once, you break it into small, manageable parts (increments). You design, build, and deploy version 1.0, then repeat the entire SDLC to deliver version 2.0.



3. The Agile Model (Iterative & Collaborative)

Agile was born out of frustration with Waterfall's rigidity. It turns the SDLC from a linear highway into a rapid, repeating loop.

- **The Workflow:** Instead of trying to build the entire product at once, Agile breaks the project into small, usable features. The team works in short, time-boxed cycles (usually 2-week "sprints"). During a single sprint, the team plans, designs, builds, tests, and deploys just *one* small slice of the application.
- **Best For:** Startups, commercial web apps, and projects where user needs and market conditions change rapidly.
- **The Catch:** It requires constant collaboration and can suffer from "scope creep" (the project growing endlessly without a clear final destination) if not managed strictly.

Summary Matrix

Methodology	Flexibility	Risk Level	Stakeholder Involvement	Delivery Time
Waterfall	Very Low	High (errors caught late)	High at start/end	Long (months/years)
V-Model	Low	Medium (highly structured QA)	High at start/end	Long
Spiral	Medium	Low (heavy risk focus)	Continuous	Medium to Long
Agile	Exceptionally High	Low (constant validation)	Continuous (weekly)	Very Short (weeks)

User prompt: explain SDLC only in detail also about waterfall, Alternative and Agile\ explain me in a way that 6th grade standard student also learn

Response: Imagine you and your friends want to build the ultimate, epic treehouse.

Before you grab a hammer and start nailing boards to a tree, you need a plan. If you just guess as you go, the floor might fall through, or you might realize halfway through that you ran out of wood!

In the world of computers, software engineers use a special master plan called the **SDLC**, which stands for **Software Development Life Cycle**. It is a step-by-step guide for taking a big idea and building it perfectly without making messy mistakes.

The 6 Steps of the SDLC (The Treehouse Plan)

Every single app you use—like Roblox, YouTube, or Minecraft—goes through these exact six steps when it's being made:

1. **Planning (The Brainstorm):** You sit down and ask, *"What kind of treehouse do we want? How big should it be? Do we have enough money for materials?"*
2. **Designing (The Blueprint):** You draw a picture of the treehouse. You decide where the door goes, how many windows it will have, and what color to paint it.
3. **Coding (The Building):** This is where you actually grab the tools! For a treehouse, you saw wood and hammer nails. For an app, programmers sit at computers and write code.
4. **Testing (The Safety Check):** Before you let anyone inside, you shake the ladder and stomp on the floorboards to make sure they won't break. In software, testers look for glitches and bugs to make sure the app doesn't crash.
5. **Deployment (The Grand Opening):** You invite your friends over, open the door, and let everyone finally hang out inside your brand-new treehouse! (For an app, this is when it goes live on the App Store).
6. **Maintenance (The Upkeep):** If a storm blows a roof shingle off, or if you decide next summer that you want to add a cool zipline, you have to fix and upgrade it.

3 Different Ways to Build (The Methodologies)

Now, there are different ways to organize your team to do these steps. Let's look at the three biggest styles.

1. The Waterfall Style (The Strict Blueprint)

Imagine you write down every single detail of the treehouse on day one. You promise not to change *anything*. You follow the steps straight down, like water falling over a cliff. You build the whole thing from start to finish, and nobody is allowed inside until it is 100% done.

- **The Problem:** What if, right at the very end, you realize you accidentally built the door too small for anyone to fit through? You'd have to tear the whole thing down and start over!

2. The Agile Style (The LEGO Brick Method)

Instead of building a massive treehouse all at once, you decide to build it in tiny pieces—almost like using LEGO sets.

- In the first week, you just build a simple wooden platform. You sit on it and say, *"Hey, this is cool, but it needs a roof."*
- The next week, you build the roof. Then you say, *"Awesome, now let's add a rope ladder."*
- **Why it's great:** You get a usable treehouse right away, and you can easily change your mind and add cool new features as you go!

3. Alternative Styles (The Middle Ground)

Sometimes teams want a mix of both. One famous alternative is called the **V-Model**.

Think of it like having a "Safety Inspector" friend assigned to you the entire time. While you are *drawing* the blueprints for the roof, your friend is already writing down the safety test for the roof. You don't wait until the very end to check your work; you check your safety list at every single step along the way to make sure nothing goes wrong.

Quick Summary Game

Style	How it works	If you change your mind...
Waterfall	Do everything in one big, straight line.	Uh oh! It's super hard and expensive to fix.
Agile	Build in small, fast, repeating loops.	Super easy! You just change the plan for next week.
Alternative (V-Model)	Match every building step with a safety test.	You catch mistakes instantly before they cause trouble.

User prompt: also give me definitions of all these

Response: Here are simple, clear definitions for all the terms we just talked about. You can use these to easily remember them or even explain them to your classmates!

The Core Terms

SDLC (Software Development Life Cycle)

The master blueprint that programmers follow to build a piece of software from a starting idea to a finished, working app. It acts like a recipe book for making programs.

Requirement Specification (SRS)

The ultimate checklist of everything an app needs to do. It's like a wish list written down before building begins (e.g., *"The app must have a dark mode option"*).

Coding / Implementation

The actual building phase where programmers sit down and write the instructions (code) that tell the computer exactly what to do.

Software Bug

A mistake or error in the code that makes the app behave weirdly, freeze, or crash.

Deployment

The grand opening of the app. This is the moment developers upload the finished code to the internet or the App Store so real people can finally download and use it.

The 3 Ways to Build (Methodologies)

Waterfall Methodology

A step-by-step way of building software where you must finish one phase completely before starting the next one. You cannot go backward—just like water going down a waterfall.

Agile Methodology

A flexible, fast way of building software in small, repeating loops. Instead of building the whole app at once, you build tiny parts, test them, show them to users, and make changes as you go.

The V-Model (Alternative)

A safety-first version of Waterfall. For every single step of building the software, you design a matching test to double-check your work, making sure you catch mistakes immediately.

Sprint

A short, focused burst of teamwork (usually 1 to 2 weeks long) in Agile where the team designs, builds, and finishes a tiny, working feature of the app.

User prompt: sprint is use in Agile?

Response: Yes, absolutely! **Sprint** is one of the most important words in Agile.

Think of a sprint in sports: it is a short, fast run where you give 100% of your energy to reach a finish line quickly.

In **Agile**, a sprint works the exact same way, but with coding:

- **How long is it?** It usually lasts **1 to 2 weeks**.
- **The Goal:** Instead of trying to build a whole massive app at once, the team chooses just a few tiny features (like "make a login button" and "add a search bar") and races to finish, test, and complete them before the sprint timer runs out.

Once one sprint ends, the team takes a quick breath, chooses the next set of features, and starts the next sprint!

User prompt: The V-Model (Alternative) A safety-first version of Waterfall. For every single step of building the software, you design a matching test to double-check your work, making sure you catch mistakes immediately. eiski samjh nai aa rahe thora easy explain karo in english

Response: No problem at all! Let's make the **V-Model** super easy to understand with a real-life example.

Imagine you are baking a giant, 3-layer chocolate cake for a birthday party.

If you use regular **Waterfall**, you bake the whole cake, put frosting on it, decorate it, and only right at the very end do you take a bite to see how it tastes. If you forgot the sugar, the whole cake is ruined, and you wasted hours!

The **V-Model** stops this from happening by pairing every single cooking step with a **taste-test step before** you move on.

How the V-Model Works (Step-by-Step)

The V-Model is shaped like the letter **V**.

- On the **left side** of the **V**, you plan and build.
- On the **right side** of the **V**, you test.
- They match up perfectly across from each other!

Here is how it works in real life:

1. **Step 1 (The Left Side):** You write down the recipe: *"We need 2 cups of sugar."*
 - **The Matching Test (The Right Side):** Right away, you think: *"Okay, when the cake is done, I will taste it to make sure it is sweet enough."*
2. **Step 2 (The Left Side):** You mix the batter in the bowl.
 - **The Matching Test (The Right Side):** Right away, you dip your finger in the raw batter to taste it *before* putting it in the oven.
3. **Step 3 (The Bottom of the V):** You put the cake in the oven to bake it. (This is like writing the code).

Why is it called "Safety-First"?

Because you don't wait until the end of the project to think about testing.

While you are planning the project on day one, you are **already writing the exam questions** to test it later. You are double-checking your work at every single level, so mistakes cannot hide!

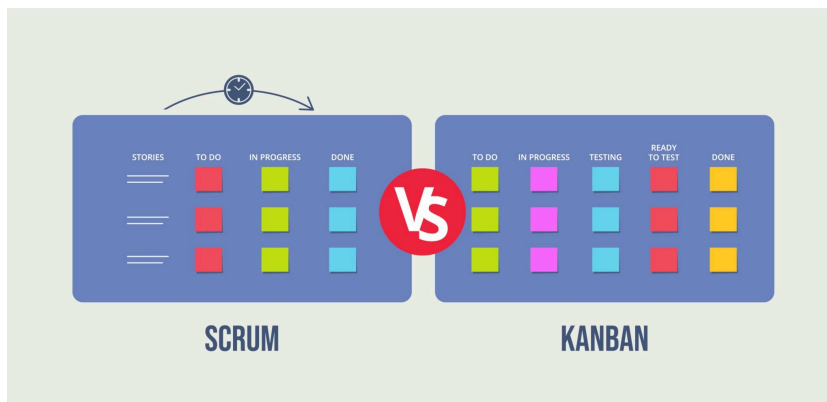
Does the cake example help make it clearer?

User prompt: what are the kanban and scrum?

Response: Now that you know what Agile is, you need to know how teams actually *do* it.

Agile is a big philosophy, but **Scrum** and **Kanban** are the two most popular "rulebooks" or styles teams use to play the Agile game.

Think of Agile as "sports" in general, while Scrum is like **soccer** (lots of strict rules, positions, and a timer) and Kanban is like **skateboarding** (very free-flowing, you go at your own pace, and you just focus on the trick right in front of you).



1. Scrum (The Team Sport)

Scrum is all about working in highly organized, fast-paced cycles. It has specific roles and strict routines.

- **The Sprint:** As we talked about, Scrum teams work in 1 to 2-week sprints. They choose a batch of work at the start and promise to finish it by the end.
- **The Roles:**
 - **The Product Owner:** The "captain" who decides *what* needs to be built (they make the list of features).
 - **The Scrum Master:** The "coach" who doesn't write code but helps the team work faster by clearing away any problems in their way.
 - **The Developers:** The players who actually build the app.
- **The Daily Standup:** A quick, 15-minute meeting every single morning. Everyone stands up (so the meeting stays short!) and answers three questions:
 1. *What did I do yesterday?*
 2. *What am I doing today?*
 3. *Is anything blocking me?*

2. Kanban (The Assembly Line)

Kanban is Japanese for "visual card." It was invented by Toyota to build cars faster, but software developers love it because it is super simple.

Instead of working in strict 2-week sprints, Kanban is like a continuous highway. There are no timers. You just focus on a visual board.

- **The Kanban Board:** A big board divided into columns—usually **To Do**, **Doing**, and **Done**. Every task is a sticky note (or "card") that travels from left to right as you work on it.
- **WIP Limits (Work in Progress Limits):** This is the magic rule of Kanban. You set a limit on how many things you can work on at the exact same time. For example, if your "Doing" column limit is **3**, you cannot start a 4th task until you finish one of the first three. This keeps the team focused so they don't get overwhelmed with half-finished work.

Scrum vs. Kanban: Quick Battle

Feature	Scrum	Kanban
How you work	In timed blocks (Sprints)	Continuous flow (no timers)
When things are delivered	At the end of every Sprint	Whenever a task is finished
Roles	Strict roles (Scrum Master, etc.)	No set roles required
Best For...	Teams building a brand new app	Help desks, support teams, or fixing bugs

User prompt: explain me scrum in detail

Response: Let's dive deep into **Scrum**. Since you already know it's a team sport with sprints, let's look at exactly how a real software team uses it day-to-day to build apps.

Think of Scrum like a repeating game loop in a video game. Every single loop (Sprint) follows a strict sequence, uses three special tools, and has three specific player roles.

1. The 3 Special Roles (The Lineup)

In a Scrum team, there are usually no "bosses." Instead, everyone has a specific job to help the team win.

- **The Product Owner (The Visionary):** This person talks to the users and customers to figure out what features the app needs. They create the ultimate "To-Do List" for the app and prioritize it so the most important things get built first.
- **The Scrum Master (The Shield):** This person is the team coach. Their job isn't to write code or tell people what to do. Their job is to protect the developers from distractions and clear away roadblocks (like if a developer's computer breaks, or they need access to a server).
- **The Developers (The Creators):** The programmers, designers, and testers who actually build, test, and deliver the working software.

2. The 3 Artifacts (The Scrum Tools)

Scrum teams use three specific things to keep track of their progress. In Scrum, these are called **Artifacts**.

- **1. The Product Backlog:** This is the master list of *every single feature, fix, and idea* for the app. It is constantly managed and ranked by the Product Owner.
- **2. The Sprint Backlog:** At the start of a new Sprint, the developers pull the top items out of the master list. This smaller list becomes their absolute promise of what they will finish during the current Sprint.
- **3. The Increment:** This is the end result. It is the actual, working piece of software that is completed by the end of the Sprint (like a new payment button that actually works and has zero bugs).

3. The 4 Scrum Ceremonies (The Game Loop)

Every Sprint follows four special meetings, called **Ceremonies**. This is the heart of how Scrum operates:

[Sprint Planning] → [Daily Standup (Every Day)] → [Sprint Review] → [Sprint Retrospective]

Meeting 1: Sprint Planning (At the very start)

The team gathers at the beginning of the week. The Product Owner shows them the top items on the Product Backlog. The developers discuss how much work they can realistically handle over the next 2 weeks, pull those items into their **Sprint Backlog**, and the countdown clock begins!

Meeting 2: The Daily Standup (Every single morning)

This is a lightning-fast, 15-minute sync. To keep it short, everyone literally stands up. Each developer quickly updates the team on three things:

1. What did I achieve yesterday?
2. What will I work on today?
3. Is anything blocking me or slowing me down?

Meeting 3: The Sprint Review (At the end of the Sprint)

The clock runs out, and the Sprint is over! The team holds a live demonstration. They invite stakeholders, users, or managers and say, *"Look what we built over the last two weeks!"* They actually click through the new features to prove they work perfectly.

🗨 Meeting 4: The Sprint Retrospective (The Reflection)

Before starting the next Sprint, the team sits down privately for a "Lessons Learned" meeting. They ask themselves:

- *What went really well that we should keep doing?*
- *What went wrong or slowed us down?*
- *How can we fix our teamwork so the next Sprint is even better?*

Once the Retrospective is over, the loop resets right back to Step 1 for the next Sprint!

Why do companies love Scrum so much? Because instead of waiting two years to see if a huge project succeeds or fails, Scrum lets companies see progress every 14 days. If the team makes a mistake, they only lose 2 weeks of time, not 2 years.

User prompt: in easy word tell me what is sprint?

Response: In super easy words, a **Sprint** is a **mini-mission with a timer**.

Instead of trying to build a whole massive app all at once (which could take a year), the team breaks the work down into tiny, bite-sized missions.

How a Sprint Works:

- **The Time Limit:** A sprint is usually exactly **2 weeks long**.
- **The Goal:** At the start of the 2 weeks, the team locks in a small list of tasks (like *"make the login button work"* and *"fix the profile picture bug"*).
- **The Race:** For those 2 weeks, the team focuses *only* on those specific tasks. They write the code, test it, and make sure it works perfectly before the timer hits zero.

Once the 2 weeks are up, the sprint is over! The team celebrates what they finished, takes a quick breath, and starts a brand-new sprint with a new list of tasks.

User prompt: can you explain me this diagram in easy words

Response: This image is a perfect summary map of the **Scrum Framework** and the **Sprint** cycle we just talked about!

Think of this diagram like a board game path, moving from **left to right**, that shows exactly how a team builds features for an app. Let's break it down into easy pieces:

Step 1: The Master Wish List (Product Backlog)

On the far left, you see a giant tower called the **Product Backlog**.

- **What it means:** This is the massive "To-Do List" for the whole app. It holds every single idea, feature request, and bug fix. The arrow loops back into it because new ideas are constantly being added to it.

Step 2: The Meeting & The Promise (Sprint Planning & Sprint Backlog)

Follow the arrow to the right, and you hit **Sprint Planning**, which leads to a smaller grid called the **Sprint Backlog**.

- **What it means:** The team looks at the massive master list during a meeting (Sprint Planning). They select just a few important tasks that they promise to finish over the next 2 weeks. That smaller chosen list is the **Sprint Backlog**.

Step 3: The Work Loop (1 Scrum Team & Daily Scrum)

Next is the big circle in the middle with the three people icons inside (**1 Scrum Team**).

- **What it means:** This is where the 2-week Sprint actually happens! The team is working hard inside this circle to build the features.
- Look at the small bubble labeled **Daily Scrum** sitting on top of the circle. That represents the quick 15-minute morning meeting where everyone shares what they are working on today.

Step 4: Show and Tell (Sprint Review)

Once the 2-week work loop is finished, the arrow points to the **Sprint Review**.

- **What it means:** This is the "Show & Tell" meeting. The team demonstrates their newly built features to managers or customers to prove everything works perfectly.

Step 5: The Gift Box (Increment)

Right after the review, the arrow points to a box labeled **Increment**.

- **What it means:** The box represents the finished, working product update! This new feature is packaged up and ready to be launched to the App Store for real users to enjoy.

Step 6: How to Improve (Sprint Retrospective)

Notice the arrow traveling all the way from the bottom/right up to the box at the top called **Sprint Retrospective**.

- **What it means:** Before starting the whole cycle over again, the team has a private meeting to talk about how they worked together. They figure out how to work smarter and faster in the next loop.
- Once that meeting finishes, the long arrow points all the way back to the beginning (**Sprint Planning**), and the loop starts all over again!

User prompt: okay i have donr my meeting with my supervisor they were using words like we make tickets in sprints and then tickets are assigned to different peoples what is this ?

Response: In the software world, "**Tickets**" are just a fancy word for **digital sticky notes**.

When your supervisor says, "*We make tickets in sprints and assign them to different people,*" they are explaining how they divide the work so everyone knows exactly what to do.

Here is how this works in real life:

1. What is a "Ticket"?

Instead of writing tasks on a piece of paper that could get lost, teams use software (like **Jira**, **Trello**, or **Asana**) to create digital cards called **Tickets**.

Each ticket represents **one specific task** that needs to be done.

Example of a Ticket:

- **Ticket Title:** *Add a "Forgot Password" link to the login screen.*
- **Description:** *If a user clicks this link, send them an email to reset their password.*
- **Status:** *To Do*
- **Assignee:** *Unassigned*

2. "We make tickets in sprints..."

Before a 2-week **Sprint** begins, the team meets up. They look at their big to-do list and turn those ideas into these digital tickets.

They only move the tickets they plan to finish over the next 2 weeks into the active Sprint. This keeps the team's screen clean so they only focus on current tasks, not future ones.

3. "...and then tickets are assigned to different people."

Once the tickets are ready for the sprint, the team distributes them:

- The supervisor (or the team together) will change the **"Assignee"** field on the digital ticket to a specific person's name.
- For example, **Ticket A** (designing the button) is assigned to *Sarah the Designer*.
- **Ticket B** (writing the code for the button) is assigned to *Alex the Developer*.

When Sarah finishes her design, she moves her ticket to **"Done."** Then, Alex sees his ticket is ready, writes the code, and moves his ticket to **"Done."**

In Short:

Your supervisor is saying:

"Every two weeks, we break our big goals down into small, individual digital task cards (tickets), and we give one card to each team member to work on so everyone knows exactly what their job is for the next two weeks."

User prompt: yes this company use jira

Response: That makes perfect sense! **Jira** is the absolute king of project management software. Almost every major tech company in the world uses it to keep track of their sprints.

Now that you know they use Jira, you can easily picture exactly what your supervisor is looking at on their screen.

What does Jira actually look like?

When your supervisor logs into Jira, they see a digital board that looks just like a grid of columns. It works exactly like a digital fridge where you put sticky notes:

- **To Do (or Backlog):** A column full of tickets (tasks) waiting for someone to start them.

- **In Progress (or Doing):** Tickets that team members are currently working on. When you start working on your assigned ticket, you literally drag-and-drop it into this column.
- **In Review / Testing:** Tickets that are finished but are being double-checked by someone else to make sure there are no bugs.
- **Done:** The graveyard of finished tickets!

Why this is a huge win for you

Since you now understand **SDLC**, **Agile**, **Sprints**, and **Jira**, you already speak the exact same language as your supervisor!

The next time they talk about work, you can confidently know:

1. **The Sprint** is their 2-week countdown timer.
2. **The Jira Tickets** are the specific jobs that need to be done.
3. **Assigning the Ticket** means someone's name is put on that card, and they are responsible for moving it from "To Do" to "Done" before the 2 weeks are up.

You are officially ready to navigate your team's meetings like a pro!

User prompt: explain me this full in detail in easy english

Response: Based on the detailed handbook images you provided, this is your company's official, step-by-step playbook for how they build software.

Let's break down this entire guide into simple, easy-to-understand English.

Section 1: The Scrum Framework & The 2-Week Sprint

Your company runs on a **strict 2-week Sprint cycle**. Here is exactly what happens from Day 1 to Day 10:

Day 1: Sprint Planning (2 to 4 hours)

The team meets to plan the next two weeks.

- **The Goal:** Take tasks from the big **Product Backlog** (the master wishlist) and move them into the **Sprint Backlog** (your 2-week promise).
- **What you do:** You estimate how long tasks will take, find any dependencies (tasks that rely on other tasks), ask questions, and propose solutions.
- **Setting Priorities:** Tasks are sorted into **MUST HAVE**, **GOOD TO HAVE**, and **OPTIONAL**.

Day 2 to Day 8: The Work & Daily Standups

- **The Daily Standup (5-10 mins):** Every single morning, you quickly answer three questions: *What did you do yesterday? What will you do today? Are there any blockers?*
- **Work Log Expectation:** The company expects **7 hours of productivity per day** (35 hours a week, or 70 hours per sprint). Developers must log their work daily in Jira.
- **Mid-Sprint Review (Day 5 - Optional):** A quick 30-minute check-in to make sure the team is on track.

Day 9: Ready for Delivery

Development wraps up. Developers must hand over their work to the Quality Assurance (QA) team early so testing isn't crammed into the very last day.

Day 10: The Wrap-up Meetings

- **Sprint Review (1 hour):** The team shows off the finished, working features to prove they are ready. You also discuss achievements and any pending work.
- **Sprint Retrospective (30 mins):** A private team meeting to discuss what went well, what went wrong, and how to improve for the next sprint.

Section 2: What Different "Tickets" Mean in Jira

In Jira, every single task is called an "Issue." Your company uses these specific types:

- **Epic:** A massive feature or category (like "User Accounts") that holds many smaller tasks. Rule: Every single task in Jira must belong to an Epic.
- **User Story:** A requirement written from the user's perspective. It must follow this exact format:
 - **As a:** [Type of user]
 - **I want to:** [Do some action]
 - **So that:** [I get a benefit]
- **Task:** Technical work that needs to be done behind the scenes (like setting up a database).
- **Bug:** A problem or mistake in the system.
- **Change Request:** A brand-new feature request that was *not* in the original plan (this usually costs extra money for the client).
- **Test Issue:** A bug found *internally* by your QA team during the active sprint. These have **zero hours estimated** on them so they don't look like "unplanned work" disrupting the sprint.

Section 3: How the Company Handles Bugs & Work

- **The Bug Rule:** Ideally, the team starts a sprint by fixing all known bugs first. If a new bug is found *during* a sprint, unless it is a critical emergency, it is put in the backlog to be fixed in the *next* sprint.
- **No Meeting Tickets:** You **do not** make Jira tickets for meetings, training, or admin work. Only actual development and deployment work go on the Jira board.
- **Unplanned Work:** The team always leaves a little bit of empty space in the sprint schedule for unexpected problems or things they might have estimated incorrectly.

Section 4: Testing & Quality Assurance (QA) Strategy

Your company takes testing very seriously and uses multiple methods to keep the app stable:

- **The QA Budget:** Around **15% to 20% of the sprint's effort** is dedicated to QA testing.
- **Load Testing:** Using a tool called **Apache Jmeter** to simulate thousands of users hitting the app at once to make sure the servers don't crash.

- **Regression Testing:** Re-testing old features to make sure that adding new code didn't accidentally break something that was already working.
- **Automatic Code Review:** On the last day of the sprint, an automated tool called **SonarQube** scans the developers' code to check for security flaws and messy writing.

Section 5: Security & IP Protection

To protect the client's app and ideas, the company follows strict rules:

- **Private Code:** All code is kept in paid, secure, private **GitHub** repositories.
 - **No Local Data:** Client databases, sales data, and client lists are never copied onto employees' personal clouds. Everything stays on secure **Amazon AWS** servers.
 - **No Credit Cards Stored:** To prevent theft, the app never stores credit card details. It passes them securely to professional payment processors like **Stripe** or **Braintree**.
-

User prompt: i want to save this chat how to save this ?

Response: Since different websites and apps have different buttons, here are the **three easiest ways** to save this chat right now so you don't lose any of this information:

Option 1: Save as a PDF (Best for keeping the layout and images)

This is the cleanest way because it saves everything exactly how it looks on your screen.

1. On a Computer (Windows or Mac):

- Press **Ctrl + P** (Windows) or **Cmd + P** (Mac) on your keyboard to open the print menu.
- Under **Destination**, change your printer to **"Save as PDF"** or **"Microsoft Print to PDF"**.
- Click **Save** and choose where you want to keep it on your computer.

2. On a Phone or Tablet:

- Tap your browser's menu button (usually three dots . . . or a share icon).
- Tap **Share** or **Print**.
- Choose **Save as PDF**.

Option 2: Copy and Paste (Best for editing later)

If you want to put this into your own study notes, a Google Doc, or a Word file:

1. Click and drag your mouse to highlight the text you want (or press **Ctrl + A** / **Cmd + A** to select the whole page).
2. Right-click and choose **Copy** (or press **Ctrl + C** / **Cmd + C**).
3. Open a blank document (like Word, Google Docs, or Notepad), right-click, and choose **Paste** (or press **Ctrl + V** / **Cmd + V**).

Option 3: Look for Built-in Buttons

Depending on the interface you are using to talk to me:

- Look at the top right or bottom of the screen for a **Share** button, a **Download/Export** icon, or a **Copy** button.
 - Some platforms let you share a read-only link of the chat directly to your email.
-

User prompt: nai nai yahi jo baat meri apksa sath hoi wi hai in gemini how to save this ?

Response: Oh, got it! standard Gemini interface mein is conversation ko save rakhne ke **3 sabse aasan tarike** hain taake yeh kabhi delete na ho:

Tarika 1: Pin the Chat (Sidebar mein save karna)

Gemini aapki chat ko automatically save karta hai, lekin naye chats aane par yeh niche chali jati hai. Isko pakka save karne ke liye:

1. Apne screen ke **left side (baayein taraf)** dekhein. Wahan ek menu (sidebar) hoga jahan "Recent" likha hoga.
2. Is chat ka naam wahan dikh raha hoga (jaise "SDLC and ADLC" ya "Standard Sprint Guide").
3. Us naam ke samne **3 dots (...)** par click karein.
4. **"Pin"** par click kar dein. *Ab yeh chat hamesha top par save rahegi aur kabhi ghum nahi hogi!*

Tarika 2: Export to Google Docs (Google Doc bana kar save karna)

Agar aap is poori baat-cheet ko ek document file mein save karna chahte hain:

1. Mere is response (ya kisi bhi response) ke bilkul niche dekhein.
2. Wahan ek **Share & Export** ka button hoga (ek chota sa arrow ya share icon bana hoga).
3. Us par click karein aur **"Export to Docs"** select karein.
4. Gemini aapke liye ek Google Doc file bana dega jismein hamari saari baatein likhi hongi. Aap "Open Docs" par click karke use download ya save kar sakte hain.

Tarika 3: Create a Shareable Link (Link bana kar rakhna)

Agar aap is chat ka ek link banana chahte hain taake baad mein click karke yahin se shuru kar sakein:

1. Niche diye gaye **Share & Export** icon par click karein.
2. **"Share"** select karein.
3. **"Create public link"** par click karein aur link ko copy kar ke apne paas WhatsApp ya Notepad mein save kar lein.

Aapko inme se kaun sa tarika sabse aasan laga?
